DV INTERVIEW · GLANCE SHEET 10 · PROJECT PART 3

Reading the RAM Back over AXI

Issue AR, collect LEN+1 beats on R, check RID / RRESP / RLAST on every beat, and compare against what was written.

AR handshake → R beats × (LEN+1)
RLAST only on beat LEN
write → read → compare = self-checking

1 End-to-end flow of the project

1 · Reset — all VALIDs LOW, ARESETn released on an edge

▶

2 · axi_write() — AW ‖ W, then B · BRESP = OKAY

▶

3 · axi_read() — AR, then LEN+1 R beats into a queue

▶

4 · Compare — each beat vs expected (model or write data)

▶

5 · Report — error count → PASS / FAIL

The read must start **after the B handshake** of the write it depends on. AXI gives no ordering between reads and writes, even to the same address — waiting for BRESP is how the master guarantees it reads the new data.

2 The `axi_read` task

```
task automatic axi_read(input  [ID_W-1:0]   id,
                        input  [ADDR_W-1:0] addr,
                        input  [7:0] len, input [2:0] size,
                        input  [1:0] burst,
                        output logic [DATA_W-1:0] rd_q[$]);
  rd_q.delete();
  // ---- read address channel
  ARID <= id;  ARADDR <= addr;  ARLEN <= len;
  ARSIZE <= size;  ARBURST <= burst;  ARVALID <= 1;
  do @(posedge ACLK); while (!ARREADY);
  ARVALID <= 0;

  // ---- read data channel
  for (int i = 0; i <= len; i++) begin
    do begin
      RREADY <= rand_bp ? $urandom_range(0, 1) : 1;
      @(posedge ACLK);
    end while (!(RVALID && RREADY));          // beat accepted
    rd_q.push_back(RDATA);
    if (RID   !== id)       $error("RID %0h != ARID %0h", RID, id);
    if (RRESP !== 2'b00)    $error("RRESP %b beat %0d", RRESP, i);
    if (RLAST !== (i == len)) $error("RLAST wrong at beat %0d", i);
  end
  RREADY <= 0;
endtask
```

- **Every beat** is checked for RID, RRESP and RLAST — RRESP is per beat, unlike BRESP.
- `rand_bp` = 1 randomly drops RREADY to exercise slave back-pressure handling.
- RDATA is captured on the same edge as the handshake, so it's the data the slave was holding.

3 4-beat read with one RREADY stall

`ARADDR 0x10 · ARLEN 3 · ARSIZE 010 · INCR` against the Part-1 slave.

ACLK	T1	T2	T3	T4	T5	T6	T7	T8
ARVALID	⇄	0	0	0	0	0	0	0
ARREADY	⇄	0	0	0	0	0	1	1
RVALID	0	⇄	1	⇄	⇄	⇄	0	0
RREADY	0	⇄	0	⇄	⇄	⇄	0	0
RDATA	–	D0	D1	D1	D2	D3	–	–
RLAST	0	0	0	0	0	1	0	0

- **T1** AR accepted; slave moves to R_DATA.
- **T2** D0 (mem[4]). **T3** TB drops RREADY → slave **holds D1** (address only moves on handshake).
- **T4, T5, T6** D1, D2, D3; RLAST with D3 → slave back to R_IDLE, ARREADY HIGH at T7.

With RREADY always HIGH the same read finishes one cycle earlier (T5). Run both — the stall case is where slave bugs hide.

4 Read-after-write tests & expected data (32-bit RAM)

#	Write	Read	Expected read data
1	INCR 0x10, 4 beats: 1, 2, 3, 4	INCR 0x10, LEN 3	1, 2, 3, 4
2	WRAP 0x38, 4 beats: A, B, C, D	INCR 0x30, LEN 3	**C, D, A, B** (wrap order un-rotated)
3	INCR 0x30, 4 beats: P, Q, R, S	WRAP 0x38, LEN 3	**R, S, P, Q** (rotated on read)
4	FIXED 0x40, 3 beats: X, Y, Z	FIXED 0x40, LEN 2	Z, Z, Z
5	0x11223344, then 0xAABBCCDD with WSTRB 0101	single 0x20	0x11BB33DD
6	INCR 0x00 × 16 beats (pattern i)	INCR 0x00 × 16, random RREADY	0 … 15, every RLAST correct
7	nothing written	any address	X — initialise mem or `$readmemh` first

TEST 1 IN CODE

```
wdata_q = '{32'h1, 32'h2, 32'h3, 32'h4};
wstrb_q = '{4'hF, 4'hF, 4'hF, 4'hF};
axi_write(0, 32'h10, 3, 3'b010, INCR);
axi_read (0, 32'h10, 3, 3'b010, INCR, rd_q);
foreach (rd_q[i])
  if (rd_q[i] !== wdata_q[i]) begin
    $error("beat %0d: got %h exp %h",
           i, rd_q[i], wdata_q[i]);
    errors++;
  end
```

Tests 2 and 3 are the best pair: writing one burst type and reading with another proves the **address generator** is right, not just that data loops back.

5 Narrow & concurrent reads

PICKING BYTES FROM A FULL-WIDTH BEAT

Read 1 byte at 0x22 (SIZE 000). Slave returns the whole word; master takes lane `0x22 % 4 = 2`.

RDATA [11] [BB] [33] [DD]

```
byte_val = RDATA[8*(addr % 4) +: 8];  // = 8'hBB
```

READ DURING A WRITE

```
fork
  axi_write(0, 32'h100, 7, 3'b010, INCR);
  axi_read (1, 32'h200, 7, 3'b010, INCR, rd_q);
join
```

Different addresses, both channels busy at once — proves the read and write FSMs really are independent.

! Read-side mistakes

- Starting the read before the write's **BRESP** → may read stale data.
- Checking RRESP only on the last beat (it's per beat).
- Not checking RLAST position — off-by-one bugs slip through.
- Holding RREADY HIGH forever — never tests slave back-pressure.
- Sampling RDATA without RVALID && RREADY at the same edge.
- Expecting a narrow read to return only the requested bytes — the master must pick lanes.
- Comparing against uninitialised memory (X) and calling it a pass because `!=` returns X — use `!==`.

? Interview Q&A

Question	Answer
How many R beats for ARLEN = 7?	8, RLAST on the 8th.
RRESP vs BRESP?	RRESP every beat; BRESP once per burst.
Why wait for BRESP before reading?	No R/W ordering in AXI; BRESP proves the write landed.
Why use `!==` in checks?	Catches X/Z; `!=` would give X and skip the error.
Best single test for the address logic?	Write with WRAP, read with INCR (and vice versa).
How would this become UVM?	Tasks → driver, checks → monitor + scoreboard with memory model, tests → sequences.